

i Здоровье можно было хранить в GameManager, однако он у нас занимается контролем статистики забега, и это уже было бы нарушением принципа единой ответственности. Также можно было сделать просто некий префаб объект - зону. Которая бы была на каждом уровне и сама следила бы за здоровьем текущего забега. В глобальных масштабах это могло бы способствовать удобному тестированию, но у меня тут маленький пэт-проект, и я решил сделать проще. Создать отдельный менеджер под здоровье. Он сам будет обновляться каждый этаж и его легко будет модифицировать относительно предметов на здоровье.

Я всё-таки думал оставить некую зону, которая бы просто отлавливала попадающих в неё врагов и после бы вызывала нанесение урона у глобального менеджера. А потом я понял, что всё куда проще, и чтобы на уровне не добавлять каждый раз в конец эту самую зону, можно сделать иначе.

Враги содержат у себя путь Path[] (массив координат). Если враг погибает до того, как достигнет последней точки - значит игрок его уничтожил, значит с врага могут выпасть монеты и это убийство попадёт в статистику забега. А если же враг дойдёт до финальной точки маршрута, значит он дошёл до базы игрока. В этот момент, когда идти больше некуда, враг уничтожится, но вместе с этим вызовет нанесение урона базе. А также никаких монет с него выпасть не может, и в статистику уничтоженных врагов он не попадёт.

Отрывок из EnemyMovement:

```
1 private void MoveTowardsTarget()
2 {
3     Vector3 direction = (targetPoint - transform.position).normalized;
4     UpdateAnimations(direction);
5
6     transform.position = Vector3.MoveTowards(transform.position,
7         targetPoint, moveSpeed * Time.deltaTime * SpeedGameManager.Inst
8 }
```

```

9      if (Vector3.Distance(transform.position, targetPoint) < 0.05f)
10     {
11         currentZoneIndex++;
12         if (currentZoneIndex >= pathZones.Length)
13         {
14             StopMovement();
15
16             OnFinishedPath?.Invoke();
17
18             return;
19         }
20         SetNextTarget();
21     }
22 }

```

В случае, если враг достигает финальной точки - вызывается OnFinishedPath?.Invoke() событие, которое обрабатывает EnemyController:

```

1 movement.OnFinishedPath += ReachBase;
2
3 public void ReachBase()
4 {
5     movement.StopMovement();
6     enemyHealthBar.Hide();
7     ReturnToPool();
8     BaseManager.Instance.TakeDamage(stats.GetDamage());
9     if (BaseManager.Instance.CurrentHp > 0)
10    {
11        EnemySpawnerManager.Instance.NotifyEnemyKilled(WaveIndex);
12    }
13 }

```

BaseManager.

```

1 using System;
2 using UnityEngine;
3
4 public class BaseManager : Manager<BaseManager>
5 {
6     private float BaseMaxHp = 220f;
7     /*
8      * Стартовое здоровье обозначил в 220 пунктов.
9      * Возможно при финальном ребалансе значение изменится.
10    */
11     public float MaxHp { get; private set; }
12     public float CurrentHp { get; private set; }
13
14     private float multiplierHP = 1;
15     private float bonusHP = 0;
16
17     public event Action<float, float> OnHealthChanged;
18     public event Action<float> OnHealthChange;
19     public event Action OnBaseDestroyed;
20
21     protected override void Awake()
22     {
23         base.Awake();
24         ResetBase();
25     }
26
27     /*
28      * В Awake сразу же обновляем значение на актуальное.
29      * Вызывать такой ресет придётся каждый этап.
30    */

```

```

31
32     public void ResetBase()
33     {
34         MaxHp = BaseMaxHp + bonusHP * multiplierHP;
35         CurrentHp = MaxHp;
36         OnHealthChanged?.Invoke(CurrentHp, MaxHp);
37     }
38
39     /*
40     MaxHp вычисляется через базовое значение, с прибавленными бонусными п
41     и умноженное на множитель. Поясню: возможно в игре будет предмет
42     в духе: +20 к максимальному здоровью.
43     Можно было остановиться на таком варианте, но лучше сделать с запасом,
44     под предметы: максимальное здоровье больше на 15%.
45     Не забываем вызвать событие и раздать подписчикам данные о здоровье.
46     HUD менеджер поймает данные и обновит UI текст со здоровьем.
47     */
48
49     public void TakeDamage(float damage)
50     {
51         CurrentHp -= damage;
52         CurrentHp = Mathf.Max(0f, CurrentHp);
53
54         OnHealthChanged?.Invoke(CurrentHp, MaxHp);
55
56         if (CurrentHp <= 0f)
57         {
58             OnBaseDestroyed?.Invoke();
59         }
60     }
61     /*
62     При получении урона выполняем простое вычисление,
63     проверяем выживаемость и уведомляем подписчиков. К слову, здесь
64     в будущем тоже может появиться логика, к примеру от предмета:
65     полученный урон либо делиться на два, либо умножается на два, с шансом
66     Тогда нам понадобятся флаги и новая логика, которая будет разрастаться
67     с каждым новым предметом на здоровье.
68     */
69     public void AddMultiplierHP(float multiplier)
70     {
71         multiplierHP += multiplier;
72     }
73     public void RemoveMultiplierHP(float multiplier)
74     {
75         multiplierHP -= multiplier;
76     }
77
78     public void AddBonusHP(float bonus)
79     {
80         bonusHP += bonus;
81     }
82     public void RemoveBonusHP(float bonus)
83     {
84         bonusHP -= bonus;
85     }
86 }

```

WalletManager.

i Добавляем менеджер кошелька, чтобы у игрока был бюджет, на который он сможет покупать башни и юнитов.

Код WalletManager практически такой же, единственное, у нас тут появляется Update() - мы каждую секунду прибавляем в кошелёк некую сумму (на старте 4). Это пассивная прибыль, но она и основная. Прибыль за уничтоженных врагов случайная, бонусная.

```
1 using System;
2 using UnityEngine;
3
4 public class WalletManager : Manager<WalletManager>
5 {
6     public int Coins { get; private set; }
7
8     public event Action<int> OnCoinsChanged;
9
10    private int baseCoins = 150;
11
12    private int baseRefillCoins = 4;
13    public int refillCoins { get; private set; }
14
15
16    private float coinTimer;
17
18    private int multiplierCoins = 1;
19    private int bonusCoins = 0;
20
21    private int bonusRefilCoins = 0;
22
23    private void Awake()
24    {
25        base.Awake();
26        ResetWallet();
27    }
28    private void Update()
29    {
30        coinTimer += Time.deltaTime * SpeedGameManager.Instance.SpeedMu
31        if (coinTimer >= 1f)
32        {
33            coinTimer -= 1f;
34            Add(refillCoins);
35        }
36    }
37
38    public void ResetWallet()
39    {
40        Coins = baseCoins + bonusCoins * multiplierCoins;
41        OnCoinsChanged?.Invoke(Coins);
42        refillCoins = refillCoins + bonusRefilCoins;
43    }
44
45
46    public void Add(int amount)
47    {
48        Coins += amount;
49        OnCoinsChanged?.Invoke(Coins);
50    }
51    public bool TrySpend(int amount)
52    {
53        if (Coins < amount) return false;
54        Coins -= amount;
55        OnCoinsChanged?.Invoke(Coins);
56        return true;
57    }
```

```

57     }
58
59     public void AddMultiplierMoney(int multiplier)
60     {
61         multiplierCoins += multiplier;
62     }
63     public void RemoveMultiplierMoney(int multiplier)
64     {
65         multiplierCoins -= multiplier;
66     }
67
68     public void AddBonusMoney(int bonus)
69     {
70         bonusCoins += bonus;
71     }
72     public void RemoveBonusMoney(int bonus)
73     {
74         bonusCoins -= bonus;
75     }
76
77     public void AddBonusRefilCoins(int bonus)
78     {
79         bonusRefilCoins += bonus;
80     }
81     public void RemoveBonusRefilCoins(int bonus)
82     {
83         bonusRefilCoins -= bonus;
84     }
85 }

```

⚠ Опять же, этим двум менеджерам предстоит разрастись, как и возможно паре других скриптов. В связи с добавлением различных предметов, которые могут менять логику игры, или быть просто множителями / бонусами.

📁 Полноценного инвентаря у нас нет. Активированные предметы хранятся в ArtefactManager.